

README

HelixQA

Status banner (round 219, 2026-05-19). Governance baseline: constitution submodule §11.4 covenant; CONST-035 anti-bluff; CONST-048 full-automation coverage; CONST-050 no-fakes + 100%-test-type coverage (15-row matrix in [docs/test-coverage.md](#)); CONST-051 submodules-as-equal-codebase + decoupling; CONST-054 dependency manifest ([helix-deps.yaml](#)); CONST-061 pre-force-push merge-first; CONST-062 + CONST-063 documentation always-sync. i18n migration rounds 112 / 205 / 214 landed (25 user-facing strings now LLM-/resource-loaded per CONST-046). New this round: [challenges/scripts/helixqa_orchestrator_challenge.sh](#) — 8-phase orchestrator-surface validator with built-in §1.1 paired mutation.

What this submodule is

HelixQA is an **anti-bluff QA orchestration framework** for cross-platform testing with real-time crash detection, step validation, evidence collection, and automated ticket generation.

Built on [digital.vasic.challenges](#) and [digital.vasic.containers](#) — both incorporated at the parent project's root per CONST-051(C) (nested own-org submodule chains forbidden; this submodule MUST NOT introduce its own `.gitmodules` entries for those repos).

The orchestrator's design centre is the §11.4 Operative Rule: **the bar for shipping is not “tests pass” but “users can use the feature.”** Every PASS HelixQA emits MUST carry positive runtime evidence captured during execution. A green summary line without that evidence is a critical defect of equal severity to a missing feature.

Features

- **Cross-platform testing:** Android, Android TV, Web, and Desktop
- **Android TV Channels testing:** Automatic detection and comprehensive testing of Android TV Home Screen Channels (default channel, category channels, Watch Next row, deep links)
- **Real-time crash detection:** ADB-based Android crash/ANR detection, browser and JVM process monitoring

- **Step-by-step validation:** Evidence collection at each test step to prevent false positives
- **YAML test banks:** QA-specific test case definitions with platform targeting, priority, and documentation references
- **Evidence collection:** Screenshots, logcat, video recording, stack traces — all centralized
- **Markdown ticket generation:** Auto-generated issue tickets with full evidence for AI fix pipelines
- **Multiple report formats:** Markdown, HTML, JSON
- **Speed modes:** Slow (debugging), Normal, Fast (CI)
- **Composable architecture:** Reuses Challenges framework for test execution and reporting

Prerequisites

- Go 1.24+
- Sibling directories:
 - ../challenges (digital.vasic.challenges)
 - ../containers (digital.vasic.containers)

Installation

```
go install digital.vasic.helixqa/cmd/helixqa@latest
```

Or build from source:

```
make build
# Binary at bin/helixqa
```

Usage

```
# Run QA pipeline
helixqa run --banks tests/banks/ --platform all

# Android-specific with device
helixqa run --banks tests/ --platform android \
  --device emulator-5554 \
  --package com.example.app

# List test cases from banks
helixqa list --banks tests/banks/ --platform android

# Generate report from existing results
helixqa report --input qa-results --format html

# Version info
helixqa version
```

Test Bank Format (YAML)

```
version: "1.0"
name: "Yole Core Tests"
test_cases:
  - id: TC-001
    name: "Create new document"
    category: functional
    priority: critical
    platforms: [android, web, desktop]
    steps:
      - name: "Open app"
        action: "Launch application"
        expected: "Main editor screen visible"
    tags: [core, smoke]
    documentation_refs:
      - type: user_guide
        section: "3.1"
        path: "docs/USER_MANUAL.md"
```

Architecture

cmd/helixqa/	CLI entry point (subcommands: run, list, report, autonomous, version)
pkg/	
config/	Configuration types and validation
testbank/	YAML test bank management with platform/
priority filtering	
detector/	Platform-specific crash/ANR detection
android.go	ADB-based detection (pidof, logcat,
screencap)	
web.go	Browser process monitoring (pgrep)
desktop.go	JVM/process monitoring (pgrep, kill)
validator/	Step-by-step validation with evidence
evidence/	Centralized evidence collection
(screenshots, video, logs)	
ticket/	Markdown ticket generation for AI fix
pipelines	
reporter/	QA report generation (reuses challenges/pkg/
report)	
orchestrator/	Main QA pipeline coordinator
autonomous/	SessionCoordinator, PlatformWorker,
PhaseManager	
navigator/	NavigationEngine, ActionExecutor (ADB,
Playwright, X11)	
issuedetector/	LLM-powered bug detection (visual, UX,
accessibility, functional)	
planning/	Test plan generation with platform-specific
test cases	
androidtv_channels_framework.go	Generic Android TV Channels

testing framework

session/

SessionRecorder, Timeline, VideoManager

See [ARCHITECTURE.md](#) and [API_REFERENCE.md](#) for details.

Autonomous QA Session

HelixQA includes an **Autonomous QA Session** mode that uses LLM-powered agents and computer vision to autonomously navigate applications, verify documented features, discover bugs, and generate comprehensive QA reports with video evidence.

What It Does

The autonomous session runs in 4 phases:

1. **Setup** – Select LLMs via LLMsVerifier, build a feature map from project docs via DocProcessor, spawn CLI agents via LLMOrchestrator, and initialize VisionEngine.
2. **Doc-Driven Verification** – Platform workers verify every documented feature against the running app, capturing screenshots and video evidence at each step.
3. **Curiosity-Driven Exploration** – Workers explore undiscovered areas of the app, testing edge cases, empty inputs, rapid interactions, and undocumented behaviors.
4. **Report & Cleanup** – Aggregate coverage, tickets, and navigation maps into a QA report (Markdown, HTML, JSON) with linked video timestamps.

New Packages

Package	Purpose
pkg/autonomous	SessionCoordinator, PlatformWorker, PhaseManager
pkg/navigator	NavigationEngine with platform-specific ActionExecutors (ADB, Playwright, X11)
pkg/issuedetector	LLM-powered bug detection across visual, UX, accessibility, and functional categories
pkg/session	SessionRecorder with video management and timeline event tracking

External Modules

The autonomous session integrates 4 external Go modules (consumed as Git submodules):

Module	Purpose
LLMsVerifier	Strategy-based LLM selection and scoring
LLMOrchestrator	Headless CLI agent management (opencode, claude-code, gemini, junie, qwen-code)
VisionEngine	GoCV mechanical vision + LLM Vision API analysis
DocProcessor	Documentation loading, feature map building, coverage tracking

CLI Subcommand

```
helixqa autonomous --project /path/to/Yole \
  --platforms android,desktop,web \
  --env .env \
  --timeout 2h \
  --coverage-target 0.9 \
  --output qa-results/ \
  --report markdown,html,json
```

Configuration

All settings are managed via a `.env` file. Copy `.env.example` to `.env` and fill in your API keys and platform-specific paths. Key configuration groups:

- **Master switch:** Enable/disable, platform selection, timeout, coverage target
- **LLMsVerifier:** Strategy, score thresholds, caching
- **API keys:** OpenAI, Anthropic, Google, Groq, Mistral, DeepSeek, xAI, Together, Qwen, Junie
- **CLI agents:** Enabled agents, binary paths, pool size, retry config
- **Vision:** Provider selection, OpenCV toggle, SSIM threshold
- **Recording:** Video/screenshot capture, ffmpeg path, quality
- **Platforms:** Android device, web URL/browser, desktop process/display

Quick Start

```
# 1. Copy and edit the configuration
cp .env.example .env
# Edit .env – set at least one API key and platform settings

# 2. Run an autonomous session against a project
helixqa autonomous --project /path/to/Yole \
  --platforms desktop \
  --env .env \
  --timeout 30m \
  --output qa-results/

# 3. View the results
cat qa-results/qa-report.md
```

```
ls qa-results/tickets/
ls qa-results/videos/
```

See [USER_GUIDE_AUTONOMOUS.md](#) and [VIDEO_COURSE_AUTONOMOUS.md](#) for detailed tutorials.

Testing

```
make test           # Run all unit + integration tests
make test-race      # With race detection
make test-cover     # With coverage report (coverage.html)
make vet            # Static analysis
make qa-all        # Full QA: every challenges/scripts/*.sh
                   # + CONST-035 anti-bluff gates
make anti-bluff     # Scan + behaviour-anchor manifest +
                   # mutation ratchet (per CONST-035)

# Round-219 addition – orchestrator-surface Challenge:
bash challenges/scripts/helixqa_orchestrator_challenge.sh
```

See [docs/test-coverage.md](#) for the full 15-row CONST-050(B) test-type matrix (unit, integration, e2e, full-automation, security, ddos, scaling, chaos, stress, performance, benchmarking, ui, ux, Challenges, autonomous-QA-session) — each row tied to an executable asset under this submodule's tree, each PASS tied to a captured-evidence shape per §11.4.2.

Test-bank conventions

Test banks under [banks/](#) are YAML (or JSON peer) documents describing platform-targeted test cases. Conventions:

- **Filename = test-bank logical name** (kebab-case <purpose>–<scope>.yaml). YAML and JSON peers share the base name (atmosphere.yaml / atmosphere.json) so tooling can auto-pair.
- **Required top-level keys:** version (string, currently "1.0"), name (human-readable), test_cases (array).
- **Per-test-case keys:** id (TC-XXX), name, category (functional / performance / security / ux / chaos / ...), priority (critical / high / medium / low), platforms (subset of [android, android_tv, web, desktop, ios, aurora_os, harmony_os]), steps[] (each with name, action, expected), tags[], optional documentation_refs[] for traceability into docs/.
- **Bank inventory floor** (round 219): ≥ 30 YAML banks on disk. Drops below the floor trigger phase 3 of the orchestrator Challenge to FAIL.
- **No hardcoded English user-facing strings in banks** per CONST-046 — name / expected strings drive LLM-generated question prompts at runtime; banks describe **structure**, not prose.

Governance pointers

Document	Authority
CONSTITUTION.md	Submodule-scoped constitutional anchors (CONST-035...063) inherited from the constitution submodule
CLAUDE.md / AGENTS.md	AI-agent operating manuals — cascade pointers + anti-bluff rules
docs/test-coverage.md	CONST-050(B) test-type coverage matrix — 15 rows, one per type
helix-deps.yaml	CONST-054 dependency manifest (Challenges + Containers, both flat-layout)
Upstreams/	CONST-056 upstream-remote recipes — <code>install_upstreams</code> reads these on clone
docs/ANTI_BLUFF.md	Anti-bluff posture details + baseline maintenance
docs/behavior-anchors.md	CONST-035 behaviour-anchor manifest (every advertised capability anchored to a test)
docs/USER_MANUAL.md	End-user-facing operating manual
USER_GUIDE_AUTONOMOUS.md / VIDEO_COURSE_AUTONOMOUS.md	Autonomous-QA-session tutorials

License

Apache-2.0